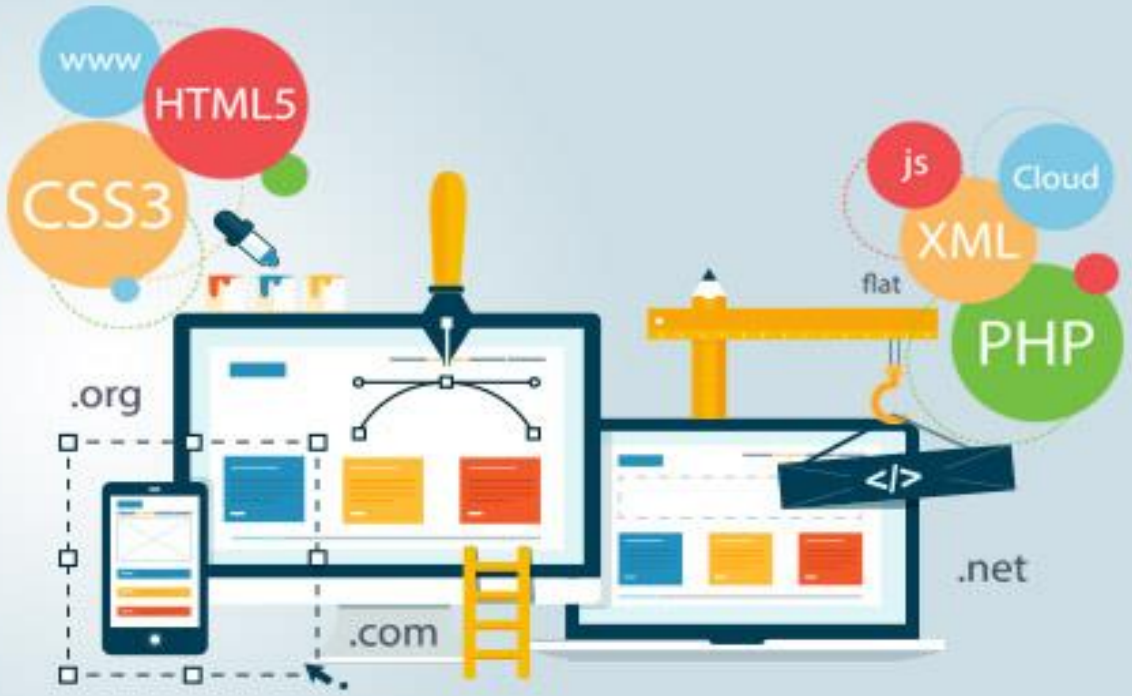
FACULTAD DE INGENIERÍA

INGENIERÍA EN SISTEMAS



Facultad de Ingeniería –UNLPam

General Pico – La Pampa – Argentina

Programación Orientada a la Web

Lic. Guillermo Javier Lafuente

Lenguaje PHP

Programación Orientada a la Web

OBJETOS

PHP no era un lenguaje OO hasta la versión 5.0

La programación OO principalmente hace uso de clases, objetos, relaciones, instancias, propiedades y métodos.

La programación **procedimental** consiste en escribir procedimientos o funciones que realizan operaciones sobre los datos, mientras que la programación **orientada a objetos** consiste en crear objetos que contienen tanto datos como funciones.

OBJETOS

Ventajas de la programación Orientada a Objetos

- La programación orientada a objetos es más rápida y fácil de ejecutar
- La programación orientada a objetos proporciona una estructura clara para los programas.
- La programación orientada a objetos ayuda a mantener el código PHP LIMPIO (o seco) "No lo repitas" y hace que el código sea más fácil de mantener, modificar y depurar.
- La programación orientada a objetos permite crear aplicaciones totalmente reutilizables con menos código y un tiempo de desarrollo más corto.

OBJETOS

La funcionalidad de un objeto esta sujeta a los datos que este maneja, una ventaja de usar objetos es que podemos modificar la funcionalidad de éste, añadir mejoras o corregir errores sin necesidad de cambiar su interfaz.

En algunas áreas de la programación de aplicaciones Web, el uso de la programación OO está desestimada, usándose una metodología estructurada basada en funciones, esto es debido a que determinados proyectos no son lo suficientemente extensos como para aplicarles una metodología OO.

OBJETOS

Las clases y los objetos son los dos aspectos principales de la programación orientada a objetos.

CLASE	OBJETOS
FRUTA	MANZANA NARANJA BANANA DURANZO

CLASE	OBJETOS
AUTO	FERRARI VOLVO BMW

OBJETOS

Supongamos que tenemos una clase llamada **Fruta**. Una fruta puede tener **propiedades** como nombre, color, peso, etc. Podemos definir variables como ***$nombre, $color y $peso*** para contener los valores de estas propiedades.

Cuando se crean los objetos individuales (manzana, plátano, etc.), heredan todas las propiedades y comportamientos de la clase, pero cada objeto tendrá valores diferentes para las propiedades.

OBJETOS – **Class**

La estructura mínima de una clase

```
Class NombreClase(){

}
```

Para que una clase sea útil, necesita atributos (propiedades) y operaciones. Podemos crear atributos como si de variables se trataran, con la palabra reservada *var (el uso de var se utilizaba para definirlas como públicas)*

```
Class NombreClase(){
  var atributo1
  var atributo2
}
```

OBJETOS – **Class**

Podemos crear métodos declarando funciones (function)

Las propiedades de clases deben ser definidas como ***public, private,*** o ***protected***. Si se declaran usando **var,** serán definidas como *public*, pero desde la vers. 5.0

```
class NombreClase {
[public | private | protected ]  $nombreVariable;

    [public | private | protected ] function metodo1() {
        }

    [public | private | protected ] function metodo2($param1,
$param2) {
        }
 }
```

Visibilidad

La visibilidad de una propiedad o método se puede definir anteponiendo una de las palabras claves **public**, **protected** o **private** en la declaración.

Miembros de clases declarados como **public** pueden ser accedidos de cualquier lado.

Miembros declarados como **protected**, sólo de la clase misma, por herencia y clases *parent*.

Aquellos definidos como **private**, únicamente de la clase que los definió.

Muchas veces no se especifica el tipo de método quedando como public

Creando una clase

```php
<?php
    class Fruit {
        // Properties
        public $name;
        public $color;

        // Methods
        function set_name($name) {
        $this->name = $name;
         }


        function get_name() {
         return $this->name;
         }
}//Cierra class fruit
?>
```

```php
$apple = new Fruit();
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple->get_name();
echo "<br>";
echo $banana->get_name();
```

Usando una clase

```php
<?php
    class Fruit {
        // Properties
          public $name;
        public $color;

        // Methods
        function set_name($name) {
          $this->name = $name;
        }
        function get_name() {
          return $this->name;
        }
        function set_color($color) {
          $this->color = $color;
        }
        function get_color() {
          return $this->color;
        }
    }
```

```php
$apple = new Fruit();
$apple->set_name('Apple');
$apple->set_color('Red');
echo "Name: " . $apple-
>get_name();
echo "<br>";
echo "Color: " . $apple-
>get_color();
?>
```

*POO_ejemplo1.php

$THIS

La palabra clave **$this** se refiere al objeto actual y solo está disponible dentro de los métodos.

Mira el siguiente ejemplo:

```php
<?php
class Fruit {
  public $name;
  function set_name($name) {
    $this->name = $name;
  }
}
$apple = new Fruit();
$apple->set_name("Apple");

echo $apple->name;
?>
```

*This1.php

```php
//cambiando el valor fuera de la clase
<?php
class Fruit {
  public $name;
}
$apple = new Fruit();
$apple->name = "Apple";

echo $apple->name;
?>
```

*this2.php

Constructores y destructores

Consejo: Como los constructores y destructores ayudan a reducir la cantidad de código, ¡son muy útiles!

comienzan con dos guiones bajos (__)!

El constructor se declara de la misma forma que los métodos, lo único que debemos tener en cuenta es que debe tener el mismo nombre que la clase.

Constructores y destructores

Un **constructor** le permite **inicializar las propiedades** de un objeto al crearlo.

Si crea una función **__construct()**, PHP llamará automáticamente a esta función cuando cree un objeto a partir de una clase.

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}
```

```php
$apple = new Fruit("Apple");
echo $apple->get_name();
?>
```

*Constructor1.php
*Constructor2.php

Constructores y destructores

Se llama a un **destructor** cuando se **destruye el objeto o se detiene o sale del script**..

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function __destruct() {
    echo "The fruit is {$this->name}.";
  }
}

$apple = new Fruit("Apple");
?>
```

* Destructor1.php

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  function __destruct() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

$apple = new Fruit("Apple", "red");
?>
```

*destructor2.php

Modificaciones de Acceso

Las propiedades y los métodos pueden tener modificadores de acceso que controlan dónde se puede acceder a ellos.

Hay tres modificadores de acceso:

público (public): se puede acceder a la propiedad o método desde cualquier lugar. Esto es predeterminado

protegido (protected): se puede acceder a la propiedad o método dentro de la clase y por clases derivadas de esa clase

privado (private): SÓLO se puede acceder a la propiedad o método dentro de la clase

Modificaciones de Acceso

En el siguiente ejemplo, hemos agregado tres modificadores de acceso diferentes a tres propiedades (nombre, color y peso). Aquí, si intenta establecer la propiedad del nombre, funcionará bien (porque la propiedad del nombre es pública y se puede acceder a ella desde cualquier lugar). Sin embargo, si intenta establecer la propiedad de color o peso, se producirá un error fatal (porque las propiedades de color y peso están protegidas y son privadas):

```php
<?php
class Fruit {
  public $name;
  protected $color;
  private $weight;
}

$mango = new Fruit();
$mango->name = 'Mango'; // OK
$mango->color = 'Yellow'; // ERROR
$mango->weight = '300'; // ERROR
?>
```

POO_acceso.php

Modificaciones de Acceso

En el siguiente ejemplo hemos agregado modificadores de acceso a dos funciones. Aquí, si intenta llamar a la función set_color() o set_weight() resultará en un error fatal (porque las dos funciones se consideran protegidas y privadas), incluso si todas las propiedades son públicas:

```php
<?php
class Fruit {
  public $name;
  public $color;
  public $weight;

  function set_name($n) {  // a public function
(default)
    $this->name = $n;
  }
  protected function set_color($n) { // a
protected function
    $this->color = $n;
  }
  private function set_weight($n) { // a private
function
    $this->weight = $n;
  }
}

$mango = new Fruit();
$mango->set_name('Mango'); // OK
$mango->set_color('Yellow'); // ERROR
$mango->set_weight('300'); // ERROR
?>
```

POO_acceso2.php

Herencia

HERENCIA

Herencia en POO = Cuando una clase deriva de otra clase.

La clase secundaria heredará todas las propiedades y métodos públicos y protegidos de la clase principal. Además, puede tener sus propias propiedades y métodos.

Una clase heredada se define utilizando la palabra clave extends. Class

```
ClaseA {
  var $atributo1;
  function operacion1(){
  }
}
class SubClaseA extends ClaseA {
  var $atributo2;
  function operacion2() {
  }
}
```

```
$x = new SubClaseA();

 $x->operacion1();
$x->atributo1 = 100;

 $x->operacion2();
$x->atributo2 = 200;
```

Herencia

Observaciones:

Como podemos observar aunque declaremos un objeto de la clase SubClaseA, al ser una clase extendida de ClaseA podemos hacer uso de todos los métodos y atributos definidos en ClaseA como si estuvieran contenidos en SubClaseA.

Debemos tener en cuenta que la herencia solo trabaja en una dirección, la subclase o clase hija hereda las características de su clase padre o superclase, pero la clase padre no posee las características de la hija. Para el caso anterior ClaseA no tendría atributo2 ni metodo2();

Herencia

```php
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color)
{
    $this->name = $name;
    $this->color = $color;
  }
  public function intro() {
    echo "The fruit is {$this->name} and the
color is {$this->color}.";
  }
}
```

```php
// Strawberry is inherited from Fruit
class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}
$strawberry = new Strawberry("Strawberry",
"red");
$strawberry->message();
$strawberry->intro();
?>
```

*herencia.php

Herencia

herencia y modificador de acceso protegido

```php
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color)
{
    $this->name = $name;
    $this->color = $color;
  }
  protected function intro() {
    echo "The fruit is {$this->name} and the
color is {$this->color}.";
  }
}
```

*herencia2.php

```php
class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}

// Try to call all three methods from outside class
$strawberry = new Strawberry("Strawberry",
"red");  // OK. __construct() is public
$strawberry->message(); // OK. message() is
public
$strawberry->intro(); // ERROR. intro() is
protected
?>
```

Constantes

- Las constantes no se pueden cambiar una vez declaradas.
- Las constantes de clase pueden resultar útiles si necesita definir algunos datos constantes dentro de una clase.
- Una constante de clase se declara dentro de una clase con la palabra clave **const**.
- Las constantes de clase distinguen entre mayúsculas y minúsculas. Sin embargo, se recomienda nombrar las constantes con letras mayúsculas.

Podemos acceder a una constante desde fuera de la clase usando el nombre de la clase seguido del operador de resolución de alcance (::) seguido del nombre de la constante, como aquí:

```
<?php
class Goodbye {
  const LEAVING_MESSAGE = “gracias a ti!";
}
echo Goodbye::LEAVING_MESSAGE;
?>
```

Constantes

Tambien podemos acceder a una constante desde dentro de la clase usando la palabra clave self seguida del operador de resolución de alcance (::) seguido del nombre de la constante, como aquí:

```
<?php
class Goodbye {
  const LEAVING_MESSAGE = “Gracias a ti!";
  public function byebye() {
    echo self::LEAVING_MESSAGE;
  }
}

$goodbye = new Goodbye();
$goodbye->byebye();
?>
```

*constante.php

Clases abstractas

¿Qué son las clases y métodos abstractos?

Las clases y métodos abstractos se dan cuando la clase principal tiene un método con nombre, pero necesita sus clases secundarias para completar las tareas.

Una clase abstracta es una clase que contiene al menos un método abstracto. Un método abstracto es un método que se declara, pero no se implementa en el código.

```php
<?php
abstract class ParentClass {
  abstract public function someMethod1();
  abstract public function someMethod2($name, $color);
  abstract public function someMethod3() : string;
}
?>
```

Clases abstractas

Al heredar de una clase abstracta, el método de la clase secundaria debe definirse con el mismo nombre y el mismo modificador de acceso o uno menos restringido. Entonces, si el método abstracto se define como protegido, el método de la clase secundaria debe definirse como protegido o público, pero no privado. Además, el tipo y la cantidad de argumentos requeridos deben ser los mismos. Sin embargo, las clases secundarias pueden tener además argumentos opcionales.

Entonces, cuando una clase hija se hereda de una clase abstracta, tenemos las siguientes reglas:

- El método de la clase secundaria debe definirse con el mismo nombre y redeclarar el método abstracto principal.
- El método de clase secundaria debe definirse con el mismo modificador de acceso o uno menos restringido.
- El número de argumentos requeridos debe ser el mismo. Sin embargo, la clase secundaria puede tener argumentos opcionales además

Clases abstractas

```php
<?php
// Parent class
abstract class Car {
  public $name;
  public function __construct($name) {
    $this->name = $name;
  }
  abstract public function intro() : string;
}

// Child classes
class Audi extends Car {
  public function intro() : string {
    return "Choose German quality! I'm an $this->name!";
  }
}

class Volvo extends Car {
  public function intro() : string {
    return "Proud to be Swedish! I'm a $this->name!";
  }
}

class Citroen extends Car {
  public function intro() : string {
    return "French extravagance! I'm a $this->name!";
  }
}
```

Clases abstractas

```php
// Create objects from the child classes
$audi = new audi("Audi");
echo $audi->intro();
echo "<br>";

$volvo = new volvo("Volvo");
echo $volvo->intro();
echo "<br>";

$citroen = new citroen("Citroen");
echo $citroen->intro();
?>
```

*clase_abstracta.php
clase_abstracta2.php

Interface - Polimorfismo

Las interfaces le permiten especificar qué métodos debe implementar una clase.

Las interfaces facilitan el uso de una variedad de clases diferentes de la misma manera. Cuando una o más clases utilizan la misma interfaz, se denomina "polimorfismo".

Las interfaces se declaran con la palabra clave interface:

```php
<?php
interface InterfaceName {
  public function someMethod1();
  public function someMethod2($name, $color);
  public function someMethod3() : string;
}
?>
```

Interface - Polimorfismo

Las interfaces vs clases abstractas

La interfaz es similar a las clases abstractas. La diferencia entre interfaces y clases abstractas es:

- Las interfaces no pueden tener propiedades, mientras que las clases abstractas sí pueden.
- Todos los métodos de interfaz deben ser públicos, mientras que los métodos de clase abstracta son públicos o protegidos.
- Todos los métodos en una interfaz son abstractos, por lo que no se pueden implementar en código y la palabra clave abstracta no es necesaria.
- Las clases pueden implementar una interfaz mientras heredan de otra clase al mismo tiempo. qué métodos debe implementar una clase.
- Las interfaces facilitan el uso de una variedad de clases diferentes de la misma manera. Cuando una o más clases utilizan la misma interfaz, se denomina "polimorfismo".

Interface - Polimorfismo

Para implementar una interfaz, una clase debe usar la palabra clave **implements**.

Una clase que implementa una interfaz debe implementar todos los métodos de la interfaz.

```php
<?php
interface Animal {
  public function makeSound();
}

class Cat implements Animal {
  public function makeSound() {
    echo "Meow";
  }
}

$animal = new Cat();
$animal->makeSound();
?>
```

*interface.php

Trait o “Rasgos”

PHP solo admite herencia única: **una clase secundaria solo puede heredar de un solo padre**.

Entonces, ¿qué pasa si una clase necesita heredar múltiples comportamientos? Los **Trait** (rasgos en ingles) de programación orientada a objetos resuelven este problema.

Los **Trait** se utilizan para declarar métodos que se pueden utilizar en varias clases. Los rasgos (traits) pueden tener métodos y métodos abstractos que se pueden usar en múltiples clases, y los métodos pueden tener cualquier modificador de acceso (público, privado o protegido).

Trait o “Rasgos”

Sintaxis

```
<?php
trait TraitName {
  // some code...
}
?>
```

Se usa con la palaba **use**

```
<?php
class MyClass {
  use TraitName;
}
?>
```

EJEMPLO

```
<?php
trait message1 {
public function msg1() {
    echo "OOP is fun! ";
  }
}

class Welcome {
  use message1;
}

$obj = new Welcome();
$obj->msg1();
?>
```

Trait.php

Trait o “Rasgos”

Aquí declaramos un rasgo: mensaje1. Luego, creamos una clase: Bienvenido. La clase usa el rasgo y todos los métodos del rasgo estarán disponibles en la clase.

Si otras clases necesitan usar la función msg1(), simplemente use el rasgo message1 en esas clases. Esto reduce la duplicación de código, porque no es necesario volver a declarar el mismo método una y otra vez.

Trait.php

Trait o “Rasgos”

Usando Mutliples trait

```php
<?php
trait message1 {
  public function msg1() {
    echo "OOP is fun! ";
  }
}

trait message2 {
  public function msg2() {
    echo "OOP reduces code duplication!";
  }
}

class Welcome {
  use message1;
}

class Welcome2 {
  use message1, message2;
}

$obj = new Welcome();
$obj->msg1();
echo "<br>";

$obj2 = new Welcome2();
$obj2->msg1();
$obj2->msg2();
?>
```

Trait2.php

Métodos static (estatico)

Los métodos estáticos se pueden llamar directamente, sin crear primero una instancia de la clase.

Los métodos estáticos se declaran con la palabra clave **static**:

Sintaxis

```php
<?php
class ClassName {
  public static function staticMethod() {
    echo "Hello World!";
  }
}
?>
```

Para acceder a un método estático utilice el nombre de la clase, dos puntos (::) y el nombre del método:

ClassName::*staticMethod*();

Métodos static (estatico)

Ejemplo

```php
<?php
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }
}

// Call static method
greeting::welcome();
?>
```

*método_estatico.php

Métodos static (estatico)

Una clase puede tener métodos tanto estáticos como no estáticos. Se puede acceder a un método estático desde un método de la misma clase usando la palabra clave self y dos puntos dobles (::):

```php
<?php
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }

  public function __construct() {
    self::welcome();
  }
}

new greeting();
?>
```

*método_estatico2.php

Los métodos estáticos también se pueden llamar desde métodos de otras clases. Para hacer esto, el método estático debe ser público.

*método_estatico3.php

Propiedades static (estatico)

Las propiedades estáticas se pueden llamar directamente, sin crear una instancia de una clase.

Las propiedades estáticas se declaran con la palabra clave estática:

Sintaxis

```php
<?php
class ClassName {
  public static $staticProp = "W3Schools";
}
?>
```

Acceso a la propiedad usando ::

ClassName::$staticProp;

```php
<?php
class pi {
  public static $value = 3.14159;
}

// Get static property
echo pi::$value;
?>
```

Propiedades static (estatico)

Una clase puede tener propiedades tanto estáticas como no estáticas. Se puede acceder a una propiedad estática desde un método de la misma clase usando la palabra clave self y dos puntos dobles (::):

```
<?php
class pi {
  public static $value=3.14159;
  public function staticValue() {
    return self::$value;
  }
}

$pi = new pi();
echo $pi->staticValue();
?>
```

Propiedades static (estatico)

Para llamar a una propiedad estática desde una clase secundaria, use la palabra clave **parent** dentro de la clase secundaria:

```php
<?php
class pi {
  public static $value=3.14159;
}
class x extends pi {
  public function xStatic() {
    return parent::$value;
  }
}
// Get value of static property directly via child class
echo x::$value;
// or get value of static property via xStatic() method
$x = new x();
echo $x->xStatic();
?>
```

Espacios de nombres

Los espacios de nombres son calificadores que resuelven dos problemas diferentes:

- Permiten una mejor organización al agrupar clases que trabajan juntas para realizar una tarea.
- Permiten utilizar el mismo nombre para más de una clase

Por ejemplo, es posible que tenga un conjunto de clases que describan una **tabla** HTML, como Tabla, Fila y Celda, y al mismo tiempo tenga otro conjunto de clases para describir muebles, como Mesa, Silla y Cama. Los espacios de nombres se pueden utilizar para organizar las clases en dos grupos diferentes y al mismo tiempo evitar que las dos clases Table y Table (mesa) se mezclen.

Espacios de nombres

sintaxis para definir el espacio de nombre html:

```
<?php
namespace Html;
?>
```

Nota: Una declaración de espacio de nombres debe ser lo primero en el archivo PHP. El siguiente código no sería válido:

```
<?php
echo "Hello World!";
namespace Html;
...
?>
```

Espacios de nombres

sintaxis para definir el espacio de nombre html:

```
<?php
namespace Html;
?>
```

Nota: Una declaración de espacio de nombres debe ser lo primero en el archivo PHP. El siguiente código no sería válido:

```
<?php
echo "Hello World!";
namespace Html;
...
?>
```

Espacios de nombres

Cree una clase de **table** en el espacio de nombres **HTML**:

```php
<?php
namespace Html;
class Table {
  public $title = "";
  public $numRows = 0;
  public function message() {
    echo "<p>Table '{$this->title}' has {$this->numRows} rows.</p>";
  }
}
$table = new Table();
$table->title = "My table";
$table->numRows = 5;
?>
```

```php
<!DOCTYPE html>
<html>
<body>

<?php
$table->message();
?>

</body>
</html>
```

*Espacio_nombres.php

Espacios de nombres

Para una mayor organización, es posible tener espacios de nombres anidados:

```php
<?php
namespace Code\Html;
?>
```

```php
<?php
include "Html.php";

$table = new Html\Table();
$table->title = "My table";
$table->numRows = 5;

$row = new Html\Row();
$row->numCells = 3;
?>

<html>
<body>

<?php $table->message(); ?>
<?php $row->message(); ?>
```

```php
<?php
namespace Html;
include "Html.php";

$table = new Table();
$table->title = "My table";
$table->numRows = 5;

$row = new Row();
$row->numCells = 3;
?>

<html>
<body>

<?php $table->message(); ?>
<?php $row->message(); ?>

</body>
</html>
```

+espacio_nombres2.php

Espacios de nombres

Alias

Podemos darle un alias al espacio de nombres para que sea mas fácil de escribir

```php
<?php
use Html as H;
$table = new H\Table();
?>
```

Una clase como alias

```php
<?php
use Html\Table as T;
$table = new T();
?>
```

```php
<?php
include "Html.php";
use Html as H;
$table = new H\Table();
$table->title = "My table";
$table->numRows = 5;
?>

<html>
<body>

<?php $table->message(); ?>

</body>
</html>
```

Alias.php

Itinerables

¿Qué es un iterable?

Un iterable es cualquier valor que se puede recorrer con un bucle foreach().

El pseudotipo iterable se introdujo en PHP 7.1 y se puede utilizar como tipo de datos para argumentos de funciones y valores de retorno de funciones.

La palabra clave iterable se puede utilizar como tipo de datos de un argumento de función o como tipo de retorno de una función:

```php
<?php
function printIterable(iterable $myIterable) {
  foreach($myIterable as $item) {
    echo $item;
  }
}
$arr = ["a", "b", "c"];
printIterable($arr);
?>
```

Itinerables

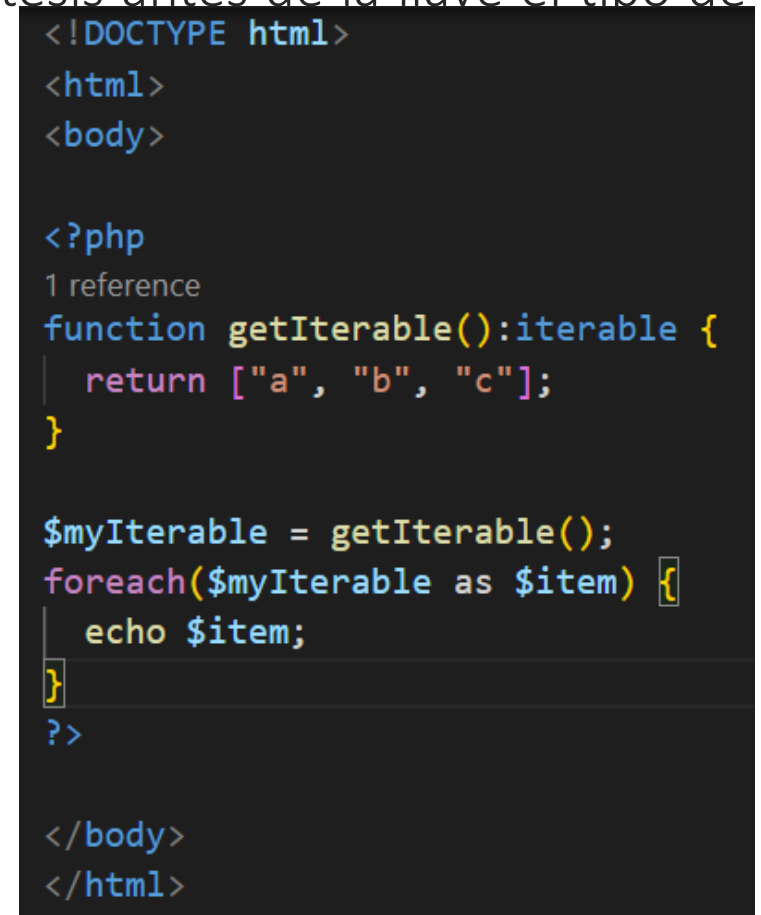
Retornando un itinerable

Se le agrega después de los paréntesis antes de la llave el tipo de dato que retorna la funcion

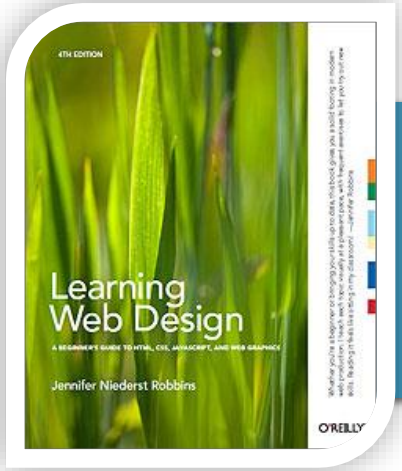
```php
<?php
function getIterable():iterable {
  return ["a", "b", "c"];
}

$myIterable = getIterable();
foreach($myIterable as $item) {
  echo $item;
}
?>
```

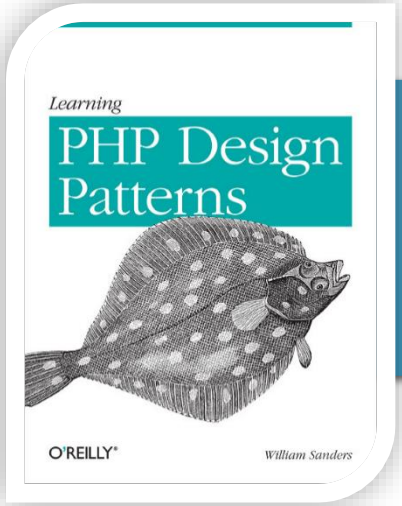
*Itinerable_retornar.php



Bibliografía



Learning Web Design
A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics (Fourth Edition)
Autor Jennifer Niederst Robbins ISBN: 978-1-449-31927-4



Learning PHP Design Patterns
William Sanders
Publisher: O'Reilly Media
Formats: Print Ebook Safari Books Online Print: February 2007
Print ISBN: 978-1-449-34491-7

Herramietas





Visual Studio Code
https://code.visualstudio.com/



Quanta Plus
http://quanta.sourceforge.net/



Dreamweaver – Adobe Macromedia ©
http://www.macromedia.com/es/software/dreamweaver/ (ver Sitio Web adobe)



Sublime Text
https://www.sublimetext.com/3/



Adobe® GoLive®
http://www.adobe.es/products/golive/main.html

Recursos educativos

Campus Virtual Facultad de Ingeniería

http://campusvirtual.ing.unlpam.edu.ar/8010/



Programación Orientada a la Web, Ago. – Dic. 2023